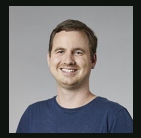
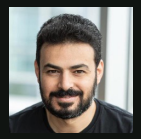
Build your own AI-assisted security workflow with GitHub Security Lab's Taskflow Agent

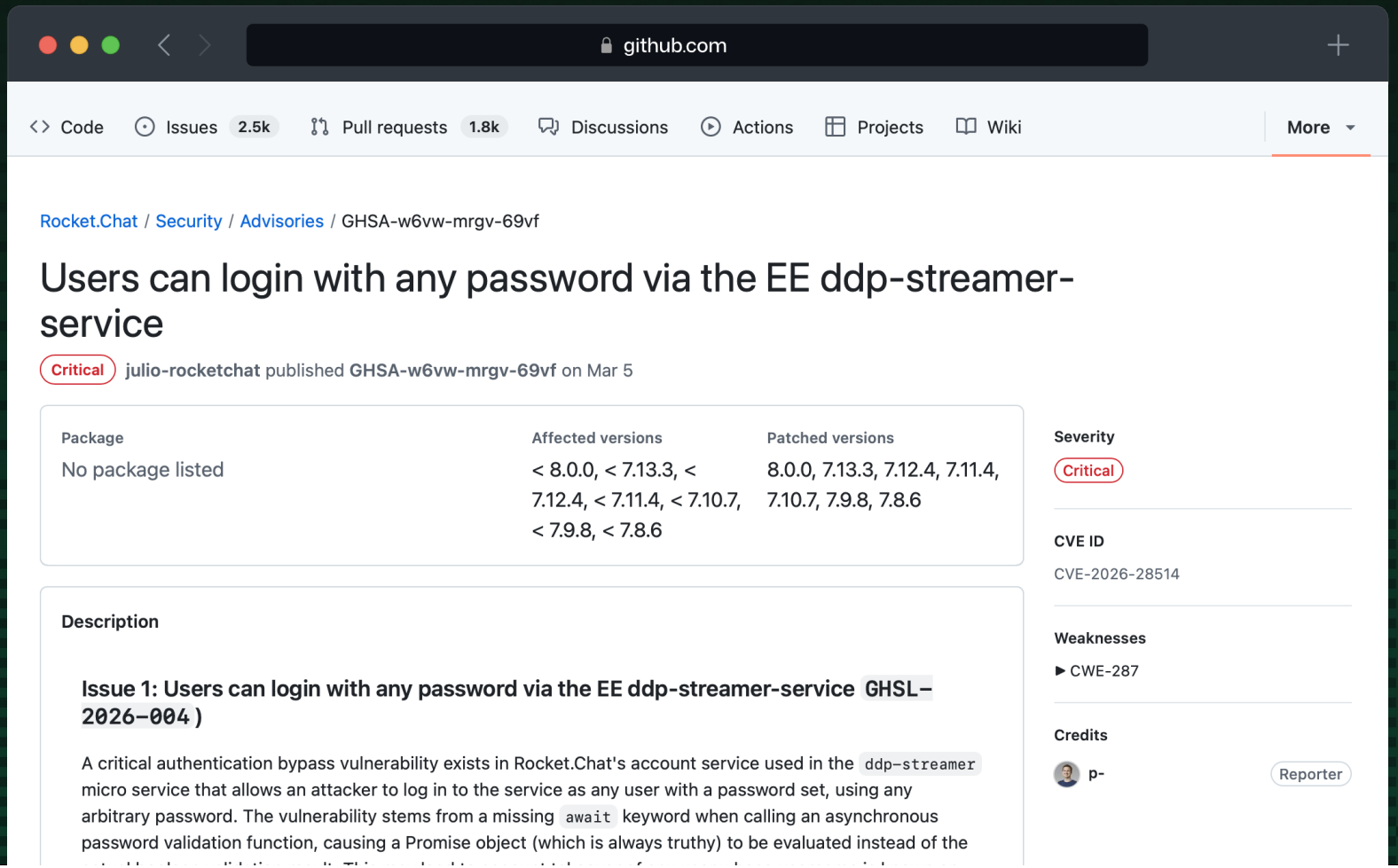


PETER STÖCKLI (P-)
Senior Security Researcher
GitHub Security Lab



KHALED YAKDAN (KYAKDAN)
Staff Security Researcher
GitHub Security Lab

A previous finding with our audit taskflows



Problem statement

Skills and prompts are great for exploring…

…but security audits need workflows that are repeatable, shareable and can be run autonomously

That’s the gap the Taskflow Agent fills.

Classified as Microsoft Confidential



Security Lab's Taskflow Agent



AI framework for security research



Designed for rapid experimentation



Uses off-the-shelf libraries



Open source: GitHub, PyPI

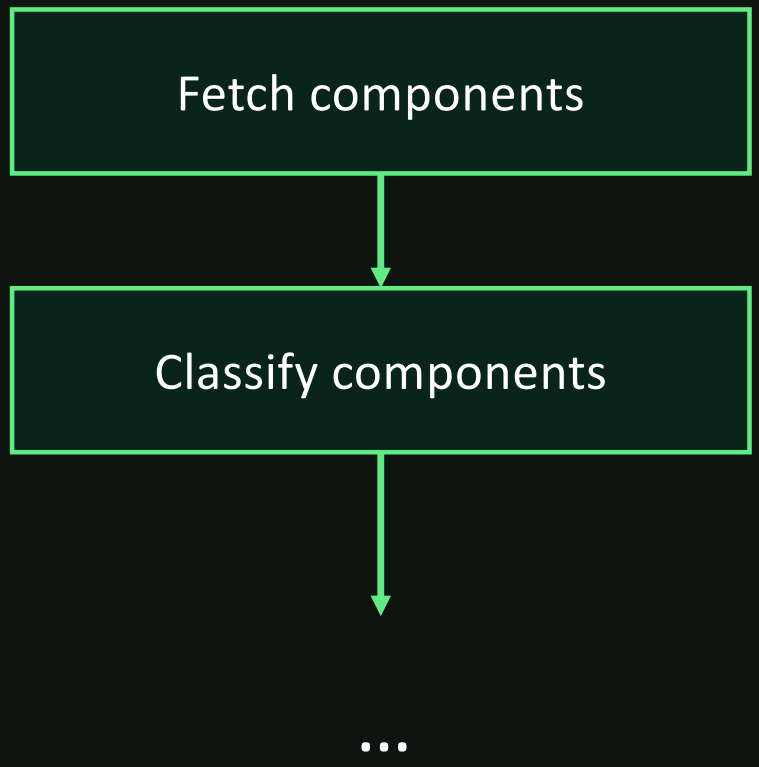


LLM Agnostic

Repository: github.com/GitHubSecurityLab/seclab-taskflow-agent

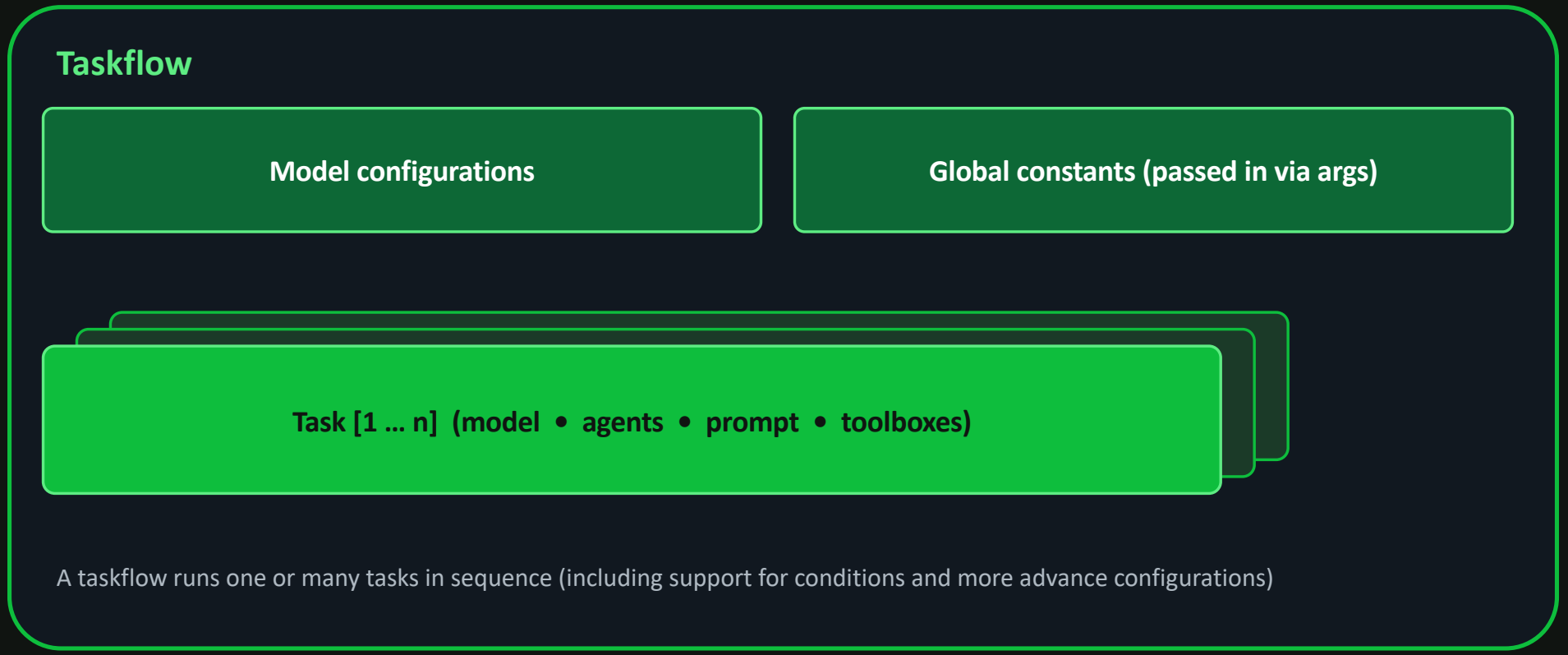
Classified as Microsoft Confidential

What is a Taskflow?



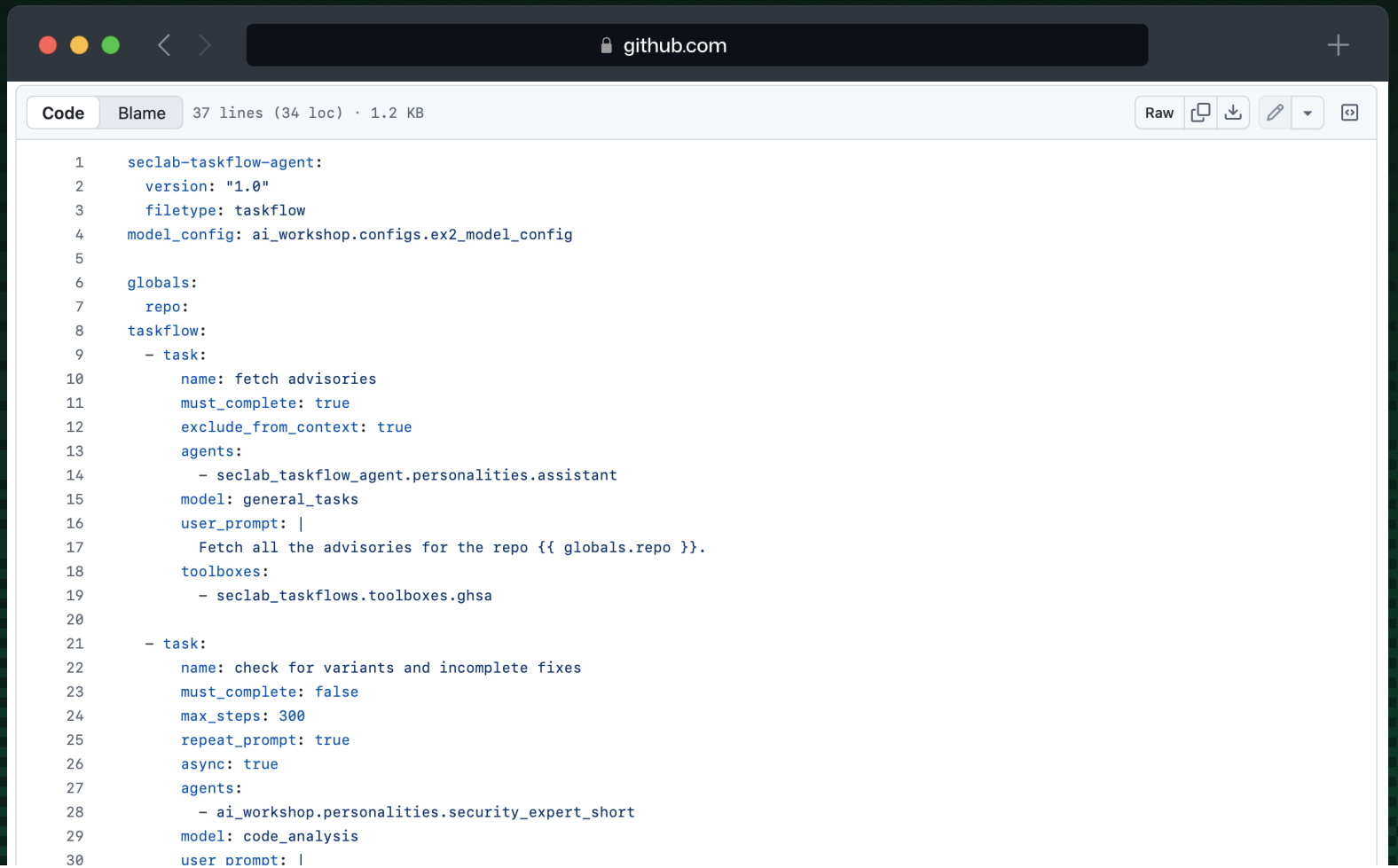
Classified as Microsoft Confidential

General anatomy of a Taskflow



Classified as Microsoft Confidential

How the Taskflow YAML looks like




```yaml
seclab-taskflow-agent:
  version: "1.0"
  filetype: taskflow
model_config: ai_workshop.configs.ex2_model_config

globals:
  repo:
taskflow:
  - task:
      name: fetch advisories
      must_complete: true
      exclude_from_context: true
      agents:
        - seclab_taskflow_agent.personalities.assistant
      model: general_tasks
      user_prompt: |
        Fetch all the advisories for the repo {{ globals.repo }}.
      toolboxes:
        - seclab_taskflows.toolboxes.ghsa

  - task:
      name: check for variants and incomplete fixes
      must_complete: false
      max_steps: 300
      repeat_prompt: true
      async: true
      agents:
        - ai_workshop.personalities.security_expert_short
      model: code_analysis
      user_prompt: |
```

Main components of a task

Key	What it does	Value / Example
`model`	Model to be used (reference to config or hard-coded)	`gpt-6-astra` `code_analysis`
`agents`	Personality: primary (first entry) and handoff agents	`seclab_taskflows.personalities.web_application_security_expert`
`user_prompt`	Prompt given to the agent	`Find all the vulnerabilities. Make no mistakes! ;)`
`toolboxes`	Defined set of MCP servers that the taskflow can use	`seclab_taskflow_agent.toolboxes.memcache` `seclab_taskflow_agent.toolboxes.codeql`
`max_steps`	Max steps to limit the number of allowed agent task loops (default 50)	`20`

Classified as Microsoft Confidential



Some available tools (MCP servers)

Container Shell

Different variants with sets of restricted tools

Network access can be removed (and typically is)

Some of our public tools

- Memcache
- CodeQL
- GitHub Actions
- GitHub Code Scanning
- GitHub File Viewer
- GitHub Security Advisories
- Local GH Resources
- Repo Context (for the audit workflows)

As part of this workshop

- Variant Report Generator
- Your own?

Classified as Microsoft Confidential



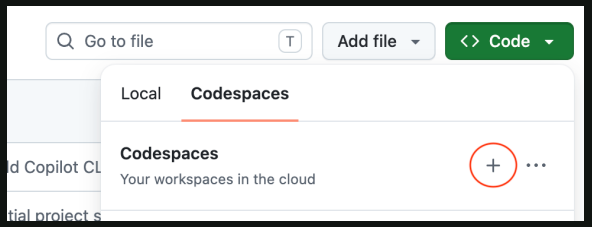
Let’s run a simple Taskflow

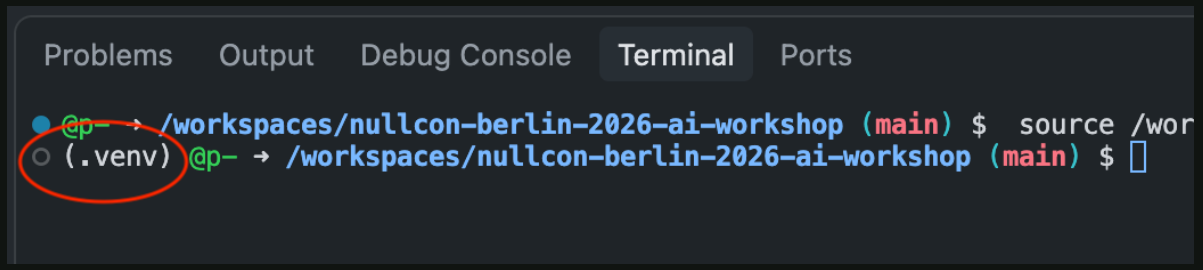
Classified as Microsoft Confidential



Run a task flow

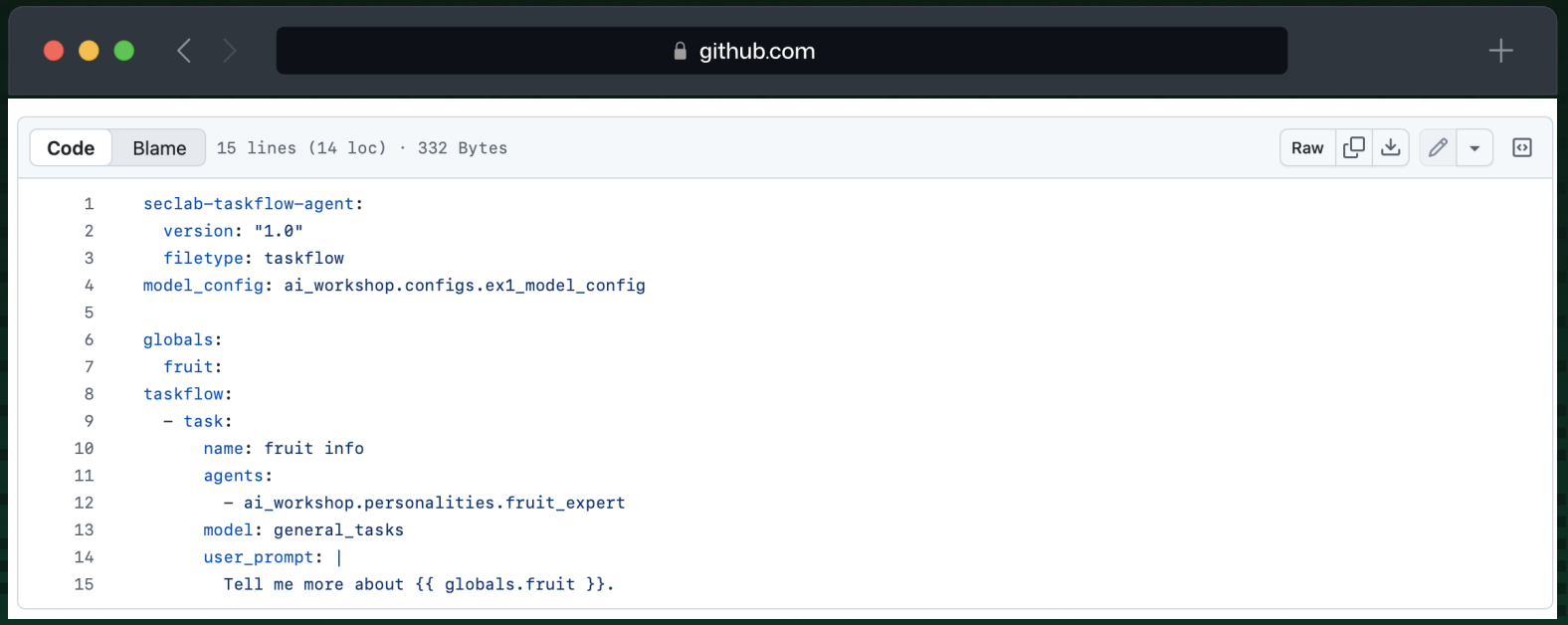
1. Go to repo at [gh.io/nc-2026-ai](gh.io/nc-2026-ai)
2. Create Codespace
3. Wait for Python venv to become active





Classified as Microsoft Confidential

A simple taskflow with globals




```yaml
seclab-taskflow-agent:
  version: "1.0"
  filetype: taskflow
model_config: ai_workshop.configs.ex1_model_config

globals:
  fruit:
taskflow:
  - task:
      name: fruit info
      agents:
        - ai_workshop.personalities.fruit_expert
      model: general_tasks
      user_prompt: |
        Tell me more about {{ globals.fruit }}.
```

Run command (in an activated Python venv):

```
python -m seclab_taskflow_agent -t ai_workshop.taskflows.ex1_globals -g fruit=apples
```

Tip: you can copy the command from the repo's README.md

The Audit Taskflows

- Audit Taskflows to discover vulnerabilities in code
- Multi-stage approach (threat modelling to issue suggestion to actual auditing)
- Taskflows are open source themselves and can be adapted to your use cases
- Run against any open source repo on GitHub (but not for free)
- The Audit Taskflows aim to find results of higher impact
- Built by the GitHub Security Lab
- On the 4th November of 2025 we had the first real findings in an open-source project

Repository: github.com/GitHubSecurityLab/seclab-taskflows

Classified as Microsoft Confidential



Challenge: Is this a vulnerability?

Imagine: you get this message:

«A function of library X executes arbitrary OS commands»

Classified as Microsoft Confidential



What is a vulnerability?

Question: Is the code execution protected, sandboxed? (e.g. GitHub Actions or any CI is command/code execution as a service.)

Question: Is is the stated goal of a method to execute commands? (e.g. it is called run_commands)

Classified as Microsoft Confidential



What is a vulnerability?

To get the complete context using SAST (Static Application Security Testing) only is pretty hard.

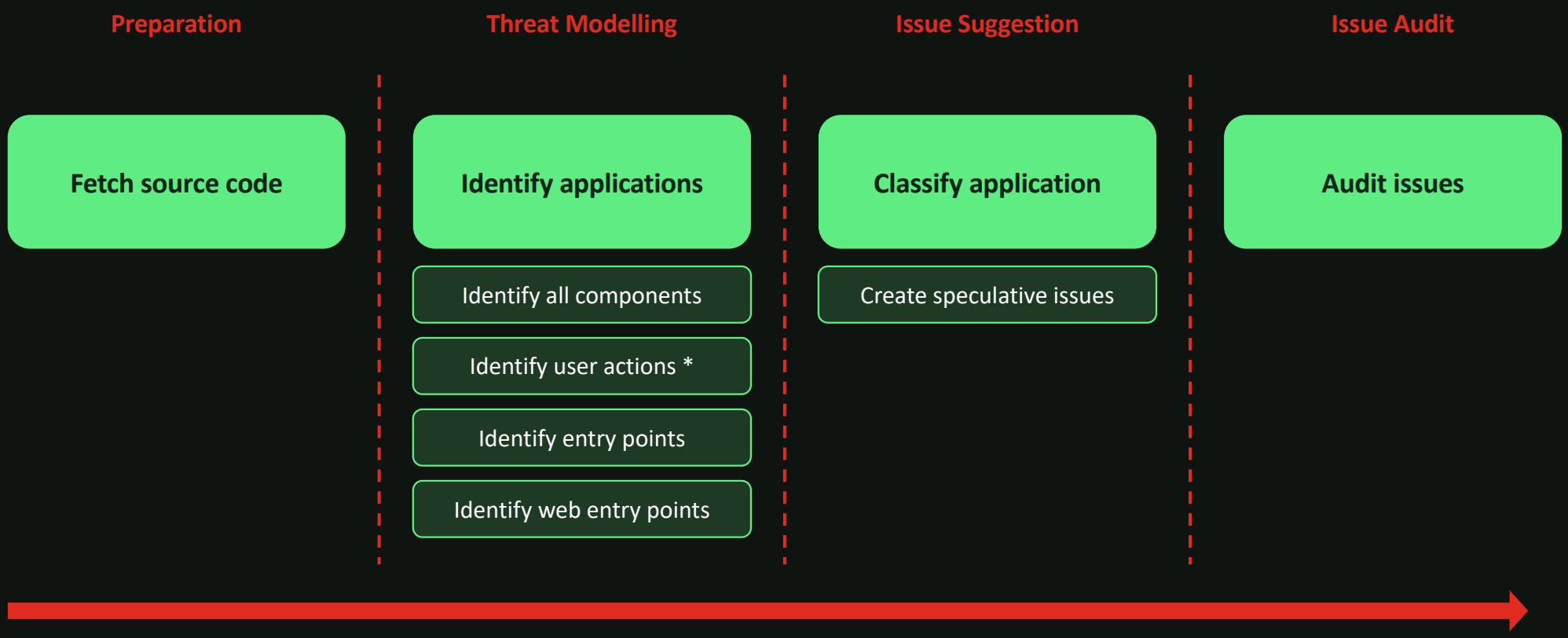
It’s much easier to do automated threat modelling using LLM and try to discover the context and the intent of certain functionalities.

But a human in the loop is still required to properly assess the severity of issues.

Classified as Microsoft Confidential



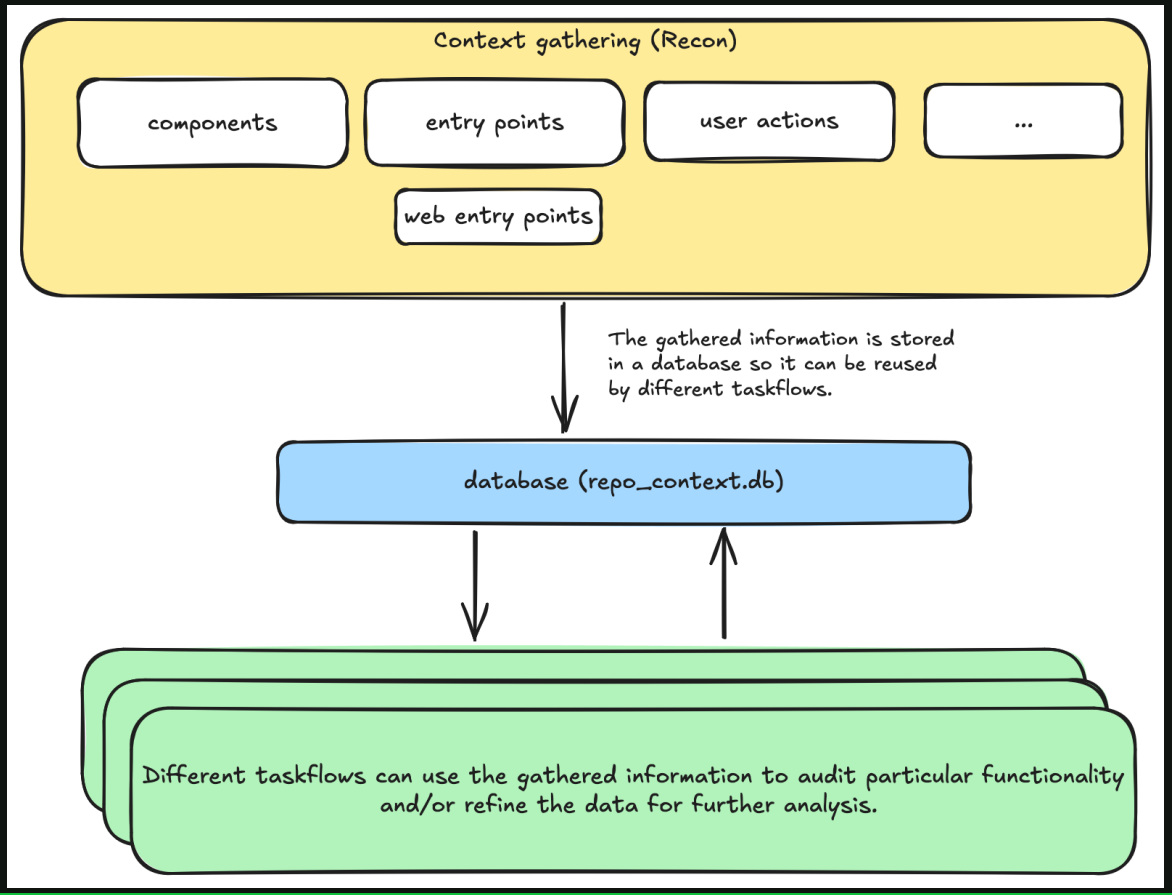
Stages of an audit run



* which actions is a user (caller) allowed to perform in respect to the intention of the component

Classified as Microsoft Confidential

The central data store (repo_context.db)



Classified as Microsoft Confidential

Findings discovered by the Audit Taskflows

Mostly high-impact bugs: RCE, authorization bypasses, IDORs, token leaks, information disclosure

Every finding is verified before it gets reported

80+ in March

reported vulnerabilities

Web audits · Mar 2026

3000+

Validated findings

Findings of various severity, findings of lower severity excluded

170

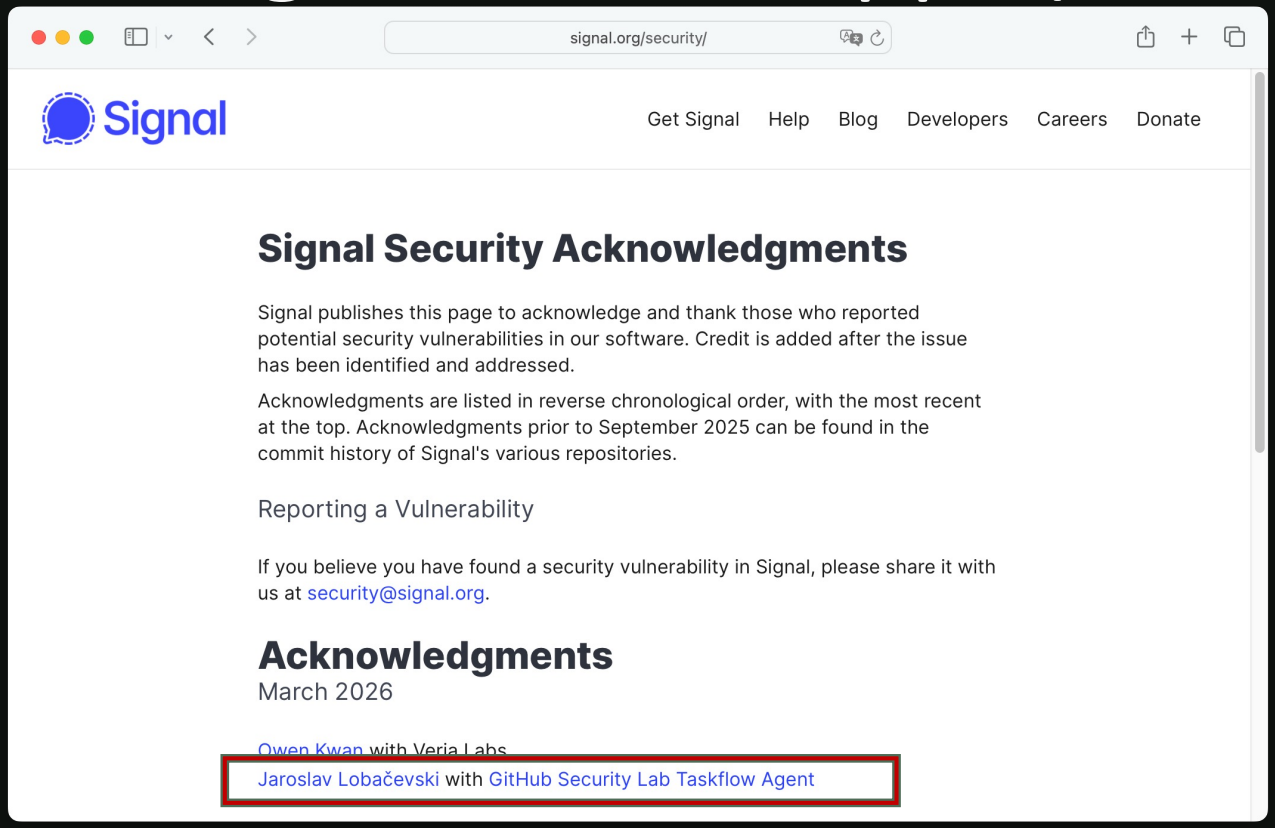
Findings delivered to SOSF projects

Directly delivered to participants of the GitHub Secure Open Source Fund

https://github.blog/security/how-to-scan-for-vulnerabilities-with-github-security-labs-open-source-ai-powered-framework/

Classified as Microsoft Confidential

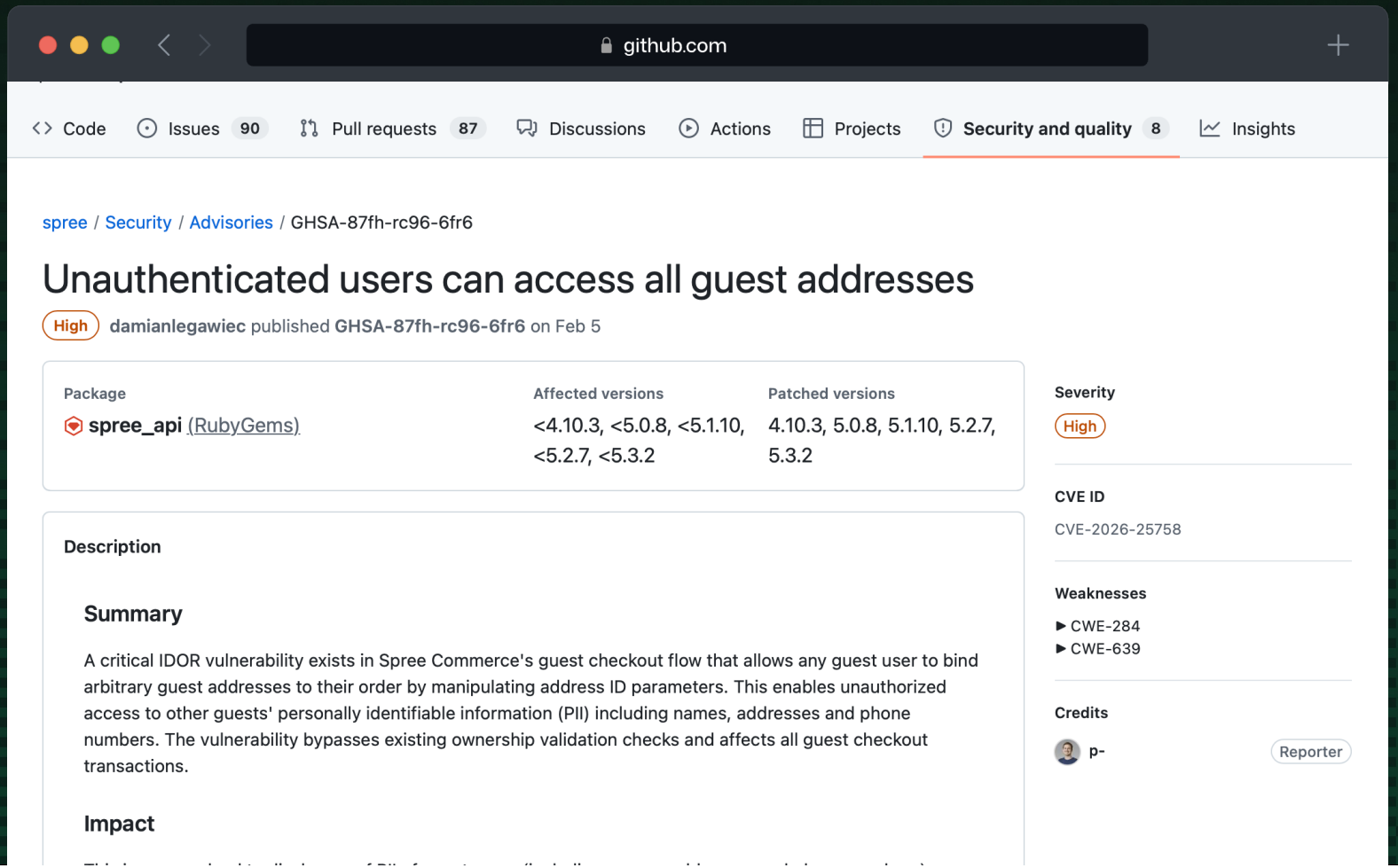
Findings in Signal mobile apps (Android & iOS)



e.g. https://securitylab.github.com/advisories/GHSL-2026-102_Android_SignalApp/

Classified as Microsoft Confidential

Exfiltrate all addresses from guest users in the Spree online shop



Example Spree finding (CVE-2026-25758)

```
11  Attack scenario (realistic):
12  1) Attacker creates a guest cart (no account) and gets an order token via cookie/param.
13  2) Attacker discovers/guesses another guest's Address.id (sequential IDs) from prior traffic or enumeration.
14  3) Attacker submits a checkout update request for their own order (using valid order_token) with e.g. order
    [bill_address_id]=VICTIM_ADDRESS_ID (or ship_address_id).
15  4) In update_from_params, id-only fields pass StrongParameters (permitted) and are applied;
    validate_address_ownership does not run on id-only fields.
16  5) In Order::AddressBook setters, because both order.user_id and target address.user_id are nil, assignment
    succeeds.
17  6) The attacker's order now references the victim's Address, exposing full name, street, phone, etc., via normal
    order views or API responses.
18
19  Why this is a vulnerability (not intended):
20  - Spree explicitly added validate_address_ownership (core/app/services/spree/checkout/update.rb) to prevent IDOR,
    but only covered nested attributes. Allowing id-only assignment to any userless Address undermines ownership and
    leaks PII.
21
22  Impact:
23  - Unauthorized read of other guests' billing/shipping address data; potential data integrity issues by linking
    orders to unrelated addresses.
```

Classified as Microsoft Confidential



Let’s have a look at the results

D E M O

(a look at the audit_result table in repo_context.db)

Classified as Microsoft Confidential



Let’s create an own research Taskflow

Classified as Microsoft Confidential



We want to find variants of past vulnerabilities or incomplete fixes based on information in advisories on GitHub (GHSAs)

What steps are required to reach that goal?

Classified as Microsoft Confidential



The first steps using existing taskflows

Fetch source code from Flask:

```
python -m seclab_taskflow_agent -t seclab_taskflows.taskflows.audit.fetch_source_code -g repo=pallets/flask
```

Fetch advisories (GHSAs) from Flask:

```
python -m seclab_taskflow_agent -t seclab_taskflows.taskflows.audit.fetch_security_advisories -g repo=pallets/flask
```

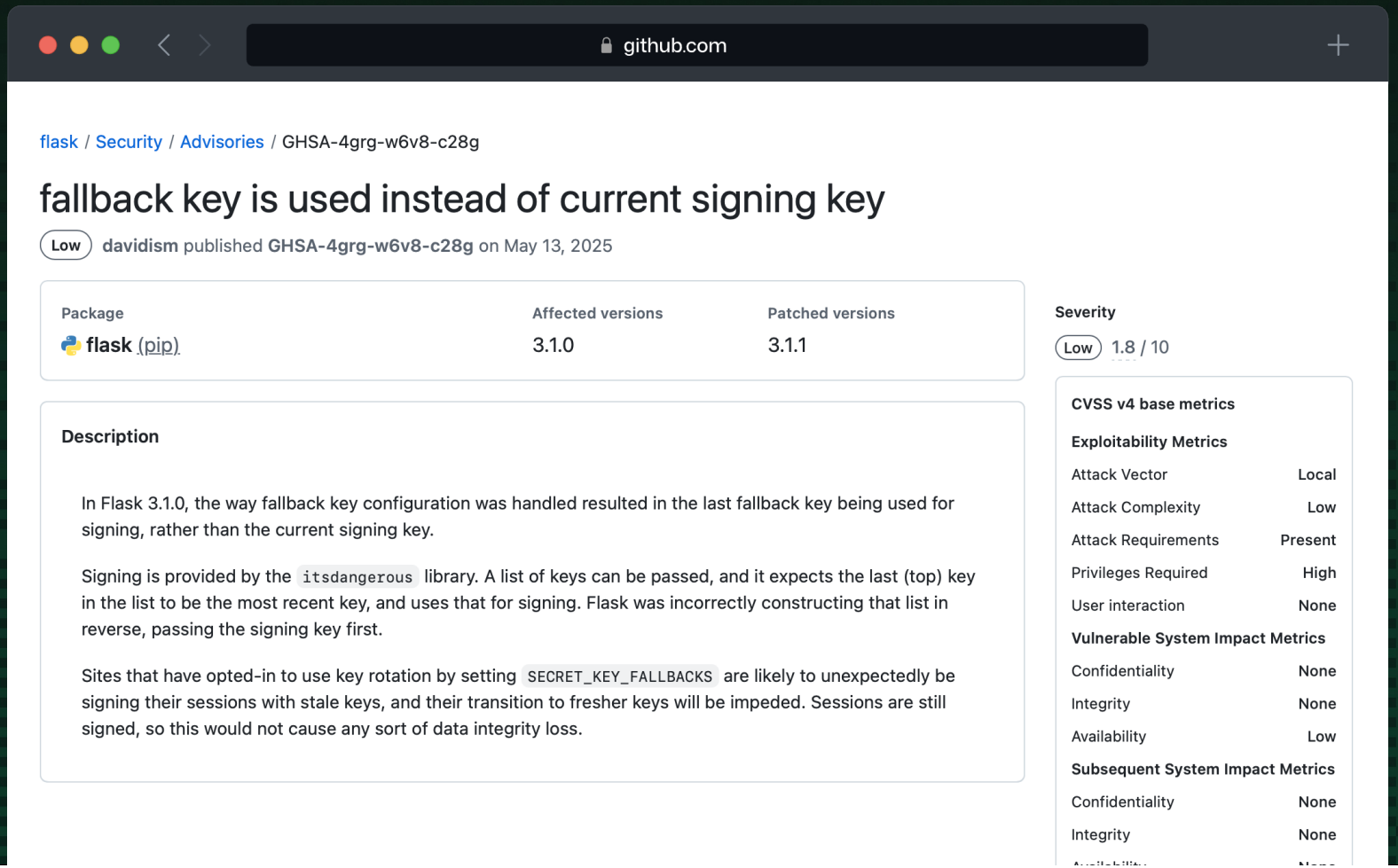
Tip: you can copy the commands from the repo's README.md

Tip: Taskflows are created to run one at a time in a CI pipeline or similar. If you need to clean up local artifacts you can execute the script in: ./scripts/clean_workshop.sh

Classified as Microsoft Confidential



Sample of Advisory: what do we need to continue?



Task: extend ex2_variant_finder.yaml + execute



After extending the Taskflow run it like this:

```
python -m seclab_taskflow_agent -t ai_workshop.taskflows.ex2_variant_finder -g repo=pallets/flask
```

Tip: you can copy the command from the repo's README.md

Let’s create an own tool using Copilot

- Open Copilot Chat and use a prompt like this to generate a new tool

> “Based on the variant_report toolbox create a new one named variant_report_json that writes the same content in JSON format. variant_exists should be reflected as boolean field.”

Classified as Microsoft Confidential



Why use Taskflows instead of Skills or Prompts

01 **Unpredictable runs**
The agent picks its own path, so results can vary from run to run with different models

02 **Uncontrolled tools**
Broad tool access is hard to restrict (e.g. no Internet access for certain steps)

03 **Lost know-how**
Security expertise stays buried in case of one-off prompts

04 **Model lock-in**
Workflows are often tied to one model, vendor or harness

05 **Context limits**
Long, multi-stage audits overflow a single context window

Classified as Microsoft Confidential



Lessons learned

One big prompt doesn’t audit too well…

…small, focused tasks with curated context do.

And a human stays in the loop.

01 **Break it down**
Drill down task by task instead of one big prompt

02 **Threat model first**
Components, entry points and user actions set realistic boundaries

03 **Suggest, then audit**
Either let a model suggest issues or suggest appropriate issues for smaller models

04 **Let the LLM filter noise**
LLM judgment discards unexploitable candidates early

05 **Don't overprompt**
Current state of the art models don't need a lot of prompting for many tasks

Classified as Microsoft Confidential

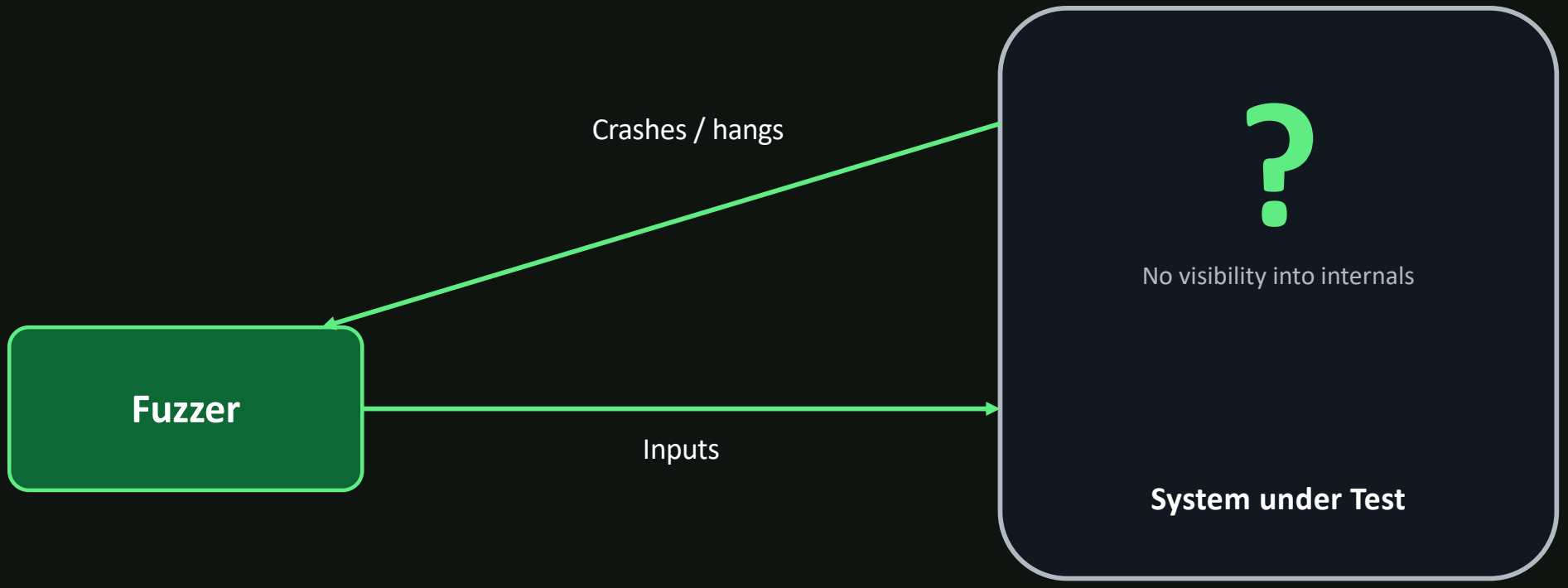


Fuzzing

Classified as Microsoft Confidential

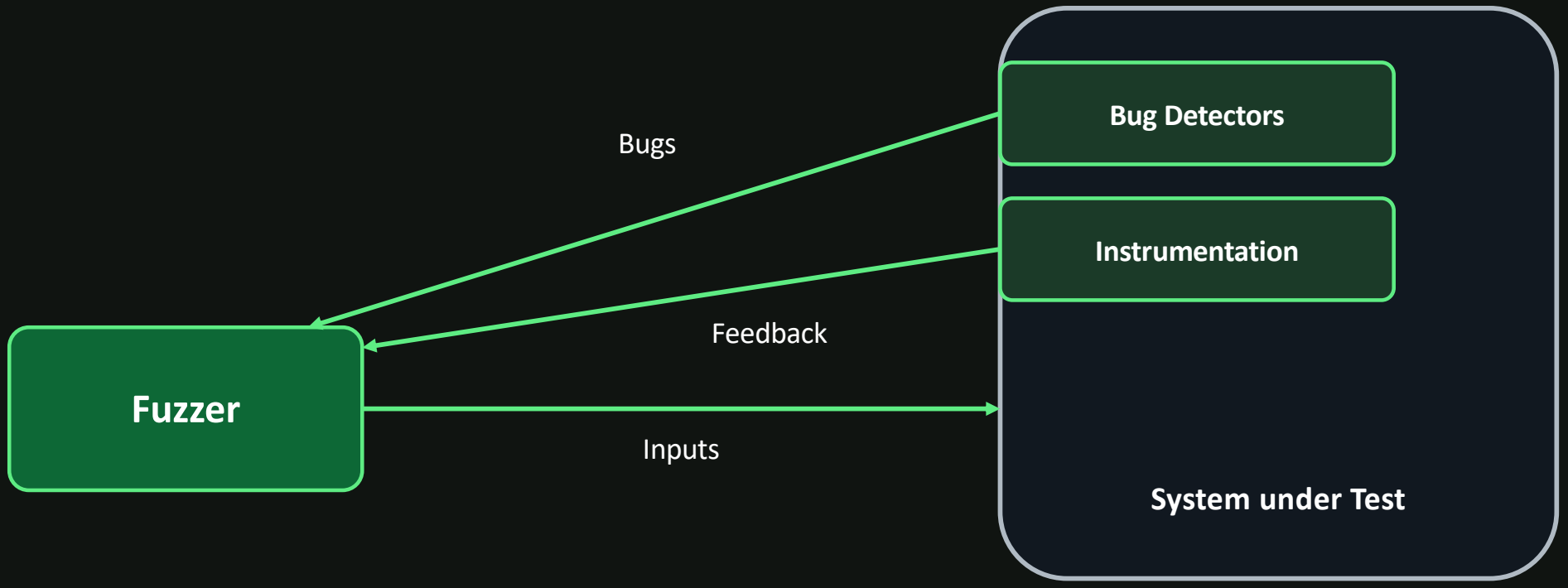


Black-box fuzzing



Classified as Microsoft Confidential

Coverage-guided fuzzing



Classified as Microsoft Confidential

Testing vs. fuzzing

Testing

```
MyApi(Input1);
MyApi(Input2);
MyApi(Input3);
```

Fuzzing

```
while (true)
  MyApi(Fuzzer.GenerateInput());
```

Classified as Microsoft Confidential



Anatomy of a fuzz harness

C++ (libFuzzer)

```
extern "C" int LLVMFuzzerTestOneInput(
    const uint8_t *data, size_t size) {

  std::string input((const char*)data, size);

  MyApi(input);
  return 0;
}
```

Raw bytes in, no result checks needed: crashes and sanitizer reports (ASan, UBSan) reveal bugs.

Java (Jazzer)

```
@FuzzTest
void fuzzTest_decode(@NotNull String input) {

  assertEquals(input,
    SomeScheme.decode(
      SomeScheme.encode(input)));
}
```

JUnit-style test: Jazzer mutates typed arguments; failed assertions and exceptions reveal bugs.

Classified as Microsoft Confidential



Exercise 1: Run a fuzz test

C (libFuzzer)

```
$ cd fuzzing/c
```

1. Build and run the fuzz test (see the README in this folder)
2. Inspect the output: crash or sanitizer report (ASan/UBSan) with stack trace

Java (Jazzer)

```
$ cd fuzzing/java
```

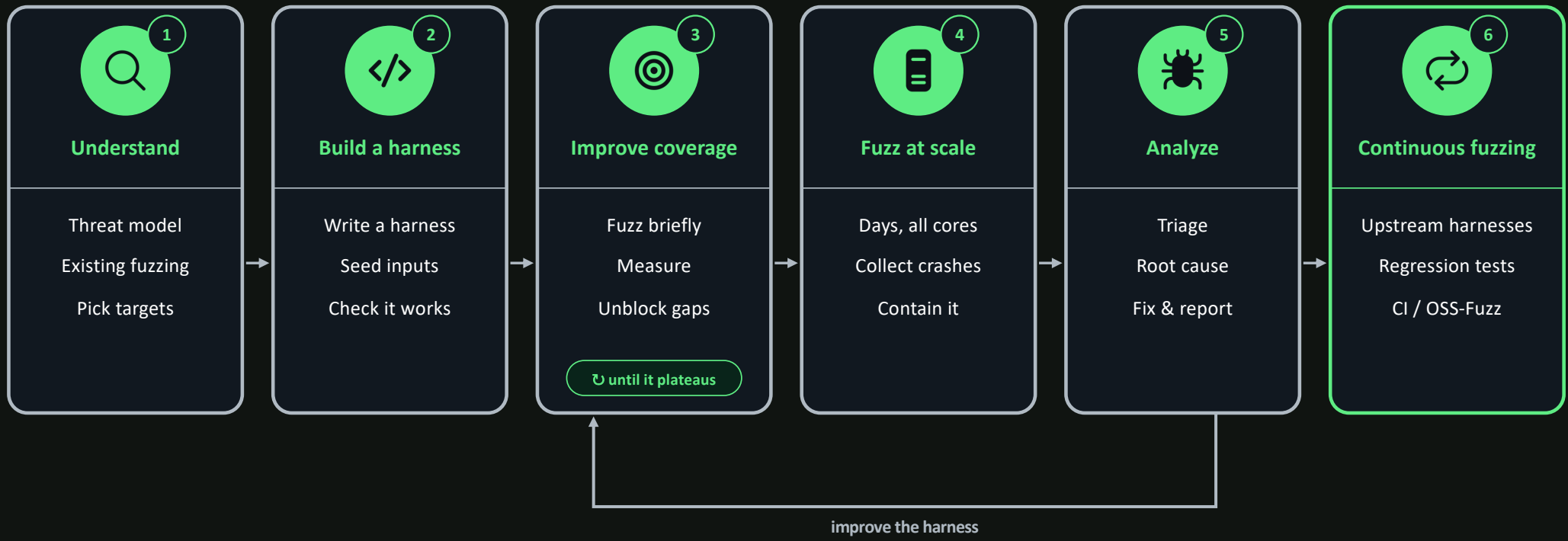
1. Build and run the fuzz test (see the README in this folder)
2. Inspect the output: failed assertion or exception with stack trace

Classified as Microsoft Confidential



The fuzzing process

Run code on millions of generated inputs to find the ones that break it



Classified as Microsoft Confidential

Agent skills for fuzzing: overview

Fuzzing know-how,

distilled for your coding agent.

github.com/GitHubSecurityLab/skills

01 **Why**

Fuzzing well is involved, and the know-how is spread across research, tools and practitioners.

02 **What**

Each step of a fuzzing campaign and how to do it well. The agent does the work; you steer.

03 **For Now**

Agents: Copilot CLI, Claude Code, Codex

Languages: C/C++, Java, JS/TS

Classified as Microsoft Confidential

The skills and how they fit together

`fuzz-campaign` Start here: “Fuzz this project”

1	2	3	4	5	6
Understand	Build a harness	Improve coverage	Fuzz at scale	Analyze	Continuous fuzzing
`security-threat-model`	`fuzz-harness`	`fuzz-coverage`	`fuzz-campaign`	`fuzz-triage`	`fuzz-campaign`

`fuzz-c-cpp` • **`fuzz-java`** • **`fuzz-js`**

Exact engine commands per language

Classified as Microsoft Confidential



Two ways to use the skills

The whole workflow

“Fuzz this project”

The agent asks at key steps

“Fuzz this project autonomously”

No questions; codespace or VM

One skill at a time

“Which APIs should I fuzz first?”

“Write a fuzz test for parse_header”

“Why is coverage stuck?”

“Triage these crashes”

Classified as Microsoft Confidential



Install the skills

Install with the Copilot CLI

```
copilot plugin marketplace add GitHubSecurityLab/skills

copilot plugin install seclab-skills@seclab

copilot skill list   # check
```

Classified as Microsoft Confidential

Exercise: fuzz a real library

Pick a track: **C/C++** **https://github.com/DaveGamble/cJSON** · **Java** **https://github.com/stleary/JSON-java**

1. Clone the selected repo into the fuzzing directory

```
$ cd fuzzing && git clone <repo-url> && cd <repo>
$ copilot --model claude-opus-5.5 --reasoning-effort medium --allow-all
```

1

“What should I fuzz here?”

Do you agree?

→

2

“Write a fuzz test for <API>”

What does it check?

Stretch: **“Fuzz this project”** on your own code

Classified as Microsoft Confidential

Contributions welcome!

The agent, Audit Taskflows, and skills are open source

Issues, new Taskflows and feedback are all appreciated

GitHub Security Lab Taskflow Agent

github.com/GitHubSecurityLab/seclab-taskflow-agent

Audit Taskflows

github.com/GitHubSecurityLab/seclab-taskflows

Agent Skills

github.com/GitHubSecurityLab/skills

Classified as Microsoft Confidential

